PUCPR • Bacharelado em Ciência da Computação

Modelagem de Sistemas Computacionais — Prof. Edson Emílio Scalabrin — 2026/2

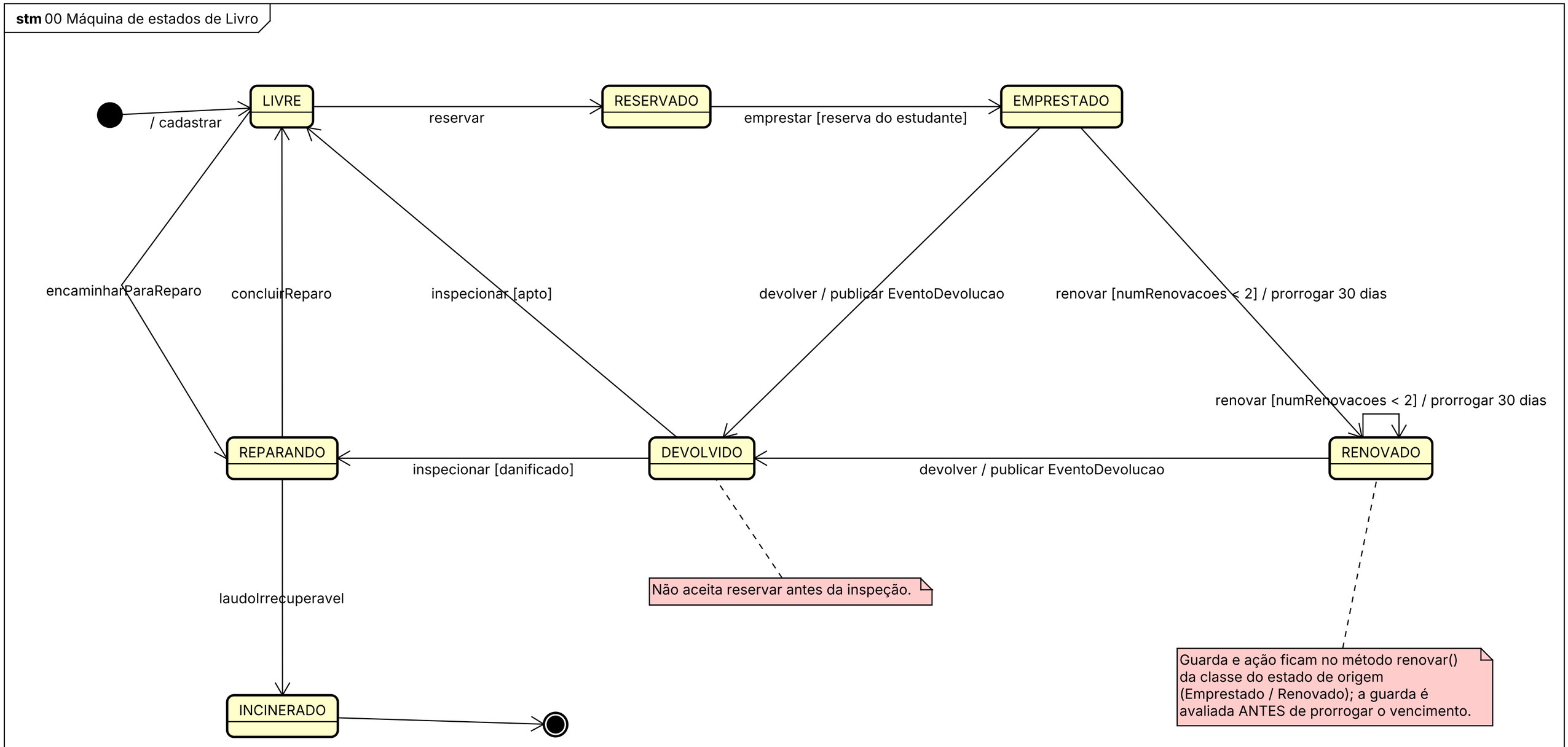
iB2B • RA03

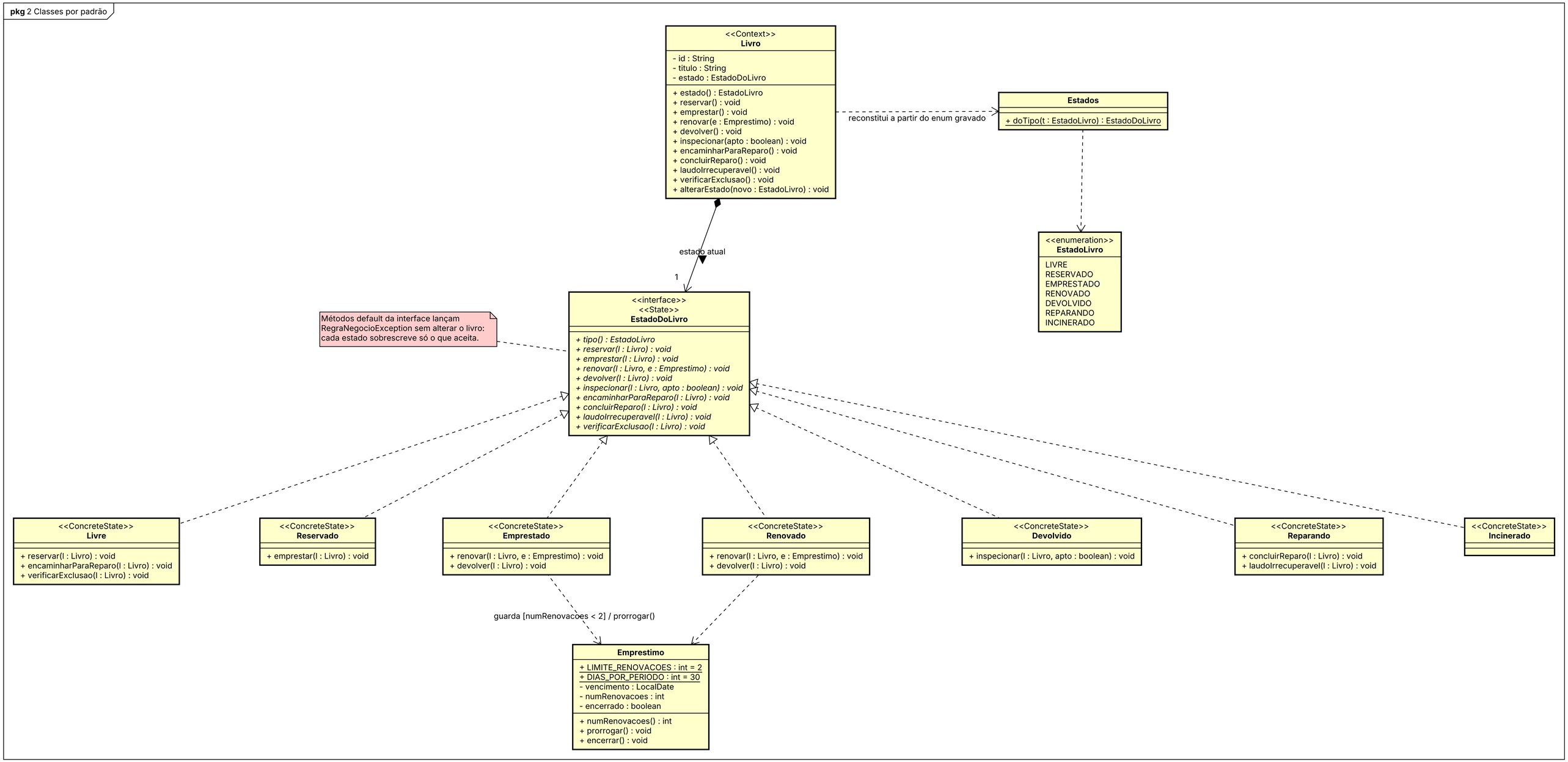
Modelos UML da implementação com dez padrões GoF

Máquina de estados de Livro • 10 diagramas de classes (um por padrão)
10 diagramas de sequência dos cenários • Implantação: tipos e instâncias

Equipe: Rafaella Somoza e Gabriela Sofia

Fonte editável: docs/uml/*.puml (PlantUML). Projeto nativo: docs/iB2B_RA03_FINAL.asta (Astah).


stm 00 Máquina de estados de Livro
LIVRE
/ cadastrar
reservar
RESERVADO
emprestar [reserva do estudante]
EMPRESTADO
encaminharParaReparo
concluirReparo
inspecionar [apto]
devolver / publicar EventoDevolucao
renovar [numRenovacoes < 2] / prorrogar 30 dias
renovar [numRenovacoes < 2] / prorrogar 30 dias
REPARANDO
inspecionar [danificado]
DEVOLVIDO
devolver / publicar EventoDevolucao
RENOVADO
Não aceita reservar antes da inspeção.
laudoIrrecuperavel
Guarda e ação ficam no método renovar()
da classe do estado de origem
(Emprestado / Renovado); a guarda é
avaliada ANTES de prorrogar o vencimento.
INCINERADO


pkg 2 Classes por padrão
<<Context>>
Livro
- id : String
- titulo : String
- estado : EstadoDoLivro
+ estado() : EstadoLivro
+ reservar() : void
+ emprestar() : void
+ renovar(e : Emprestimo) : void
+ devolver() : void
+ inspecionar(apto : boolean) : void
+ encaminharParaReparo() : void
+ concluirReparo() : void
+ laudoIrrecuperavel() : void
+ verificarExclusao() : void
+ alterarEstado(novo : EstadoLivro) : void
reconstitui a partir do enum gravado
Estados
+ doTipo(t : EstadoLivro) : EstadoDoLivro
<<enumeration>>
EstadoLivro
LIVRE
RESERVADO
EMPRESTADO
RENOVADO
DEVOLVIDO
REPARANDO
INCINERADO
estado atual
1
Métodos default da interface lançam RegraNegocioException sem alterar o livro: cada estado sobrescreve só o que aceita.
<<interface>>
<<State>>
EstadoDoLivro
+ tipo() : EstadoLivro
+ reservar(l : Livro) : void
+ emprestar(l : Livro) : void
+ renovar(l : Livro, e : Emprestimo) : void
+ devolver(l : Livro) : void
+ inspecionar(l : Livro, apto : boolean) : void
+ encaminharParaReparo(l : Livro) : void
+ concluirReparo(l : Livro) : void
+ laudoIrrecuperavel(l : Livro) : void
+ verificarExclusao(l : Livro) : void
<<ConcreteState>>
Livre
+ reservar(l : Livro) : void
+ encaminharParaReparo(l : Livro) : void
+ verificarExclusao(l : Livro) : void
<<ConcreteState>>
Reservado
+ emprestar(l : Livro) : void
<<ConcreteState>>
Emprestado
+ renovar(l : Livro, e : Emprestimo) : void
+ devolver(l : Livro) : void
<<ConcreteState>>
Renovado
+ renovar(l : Livro, e : Emprestimo) : void
+ devolver(l : Livro) : void
<<ConcreteState>>
Devolvido
+ inspecionar(l : Livro, apto : boolean) : void
<<ConcreteState>>
Reparando
+ concluirReparo(l : Livro) : void
+ laudoIrrecuperavel(l : Livro) : void
<<ConcreteState>>
Incinerado
guarda [numRenovacoes < 2] / prorrogar()
Emprestimo
+ LIMITE_RENOVACOES : int = 2
+ DIAS_POR_PERIODO : int = 30
- vencimento : LocalDate
- numRenovacoes : int
- encerrado : boolean
+ numRenovacoes() : int
+ prorrogar() : void
+ encerrar() : void

pkg 2 Classes por padrão

SistemaIB2B

escolhe por IB2B_POLITICA

<<Context>>
<<Client>>
ServicoMultas

- politica : PoliticaMulta
- banco : GatewayCobranca

+ configurarPolitica(nova : PoliticaMulta) : void
+ emitir(d : Devolucao) : Optional<BoletoMulta>
+ registrarNotificacao(msg : String) : boolean

cria com valor e nome da política

BoletoMulta

- codigo : String
- valor : BigDecimal
- politica : String
- codigoExterno : String
- quitado : boolean

+ quitar() : boolean

vigente

1

<<interface>>
<<Strategy>>
PoliticaMulta

+ nome() : String
+ calcular(diasDeAtraso : long) : BigDecimal

<<ConcreteStrategy>>
MultaOriginal

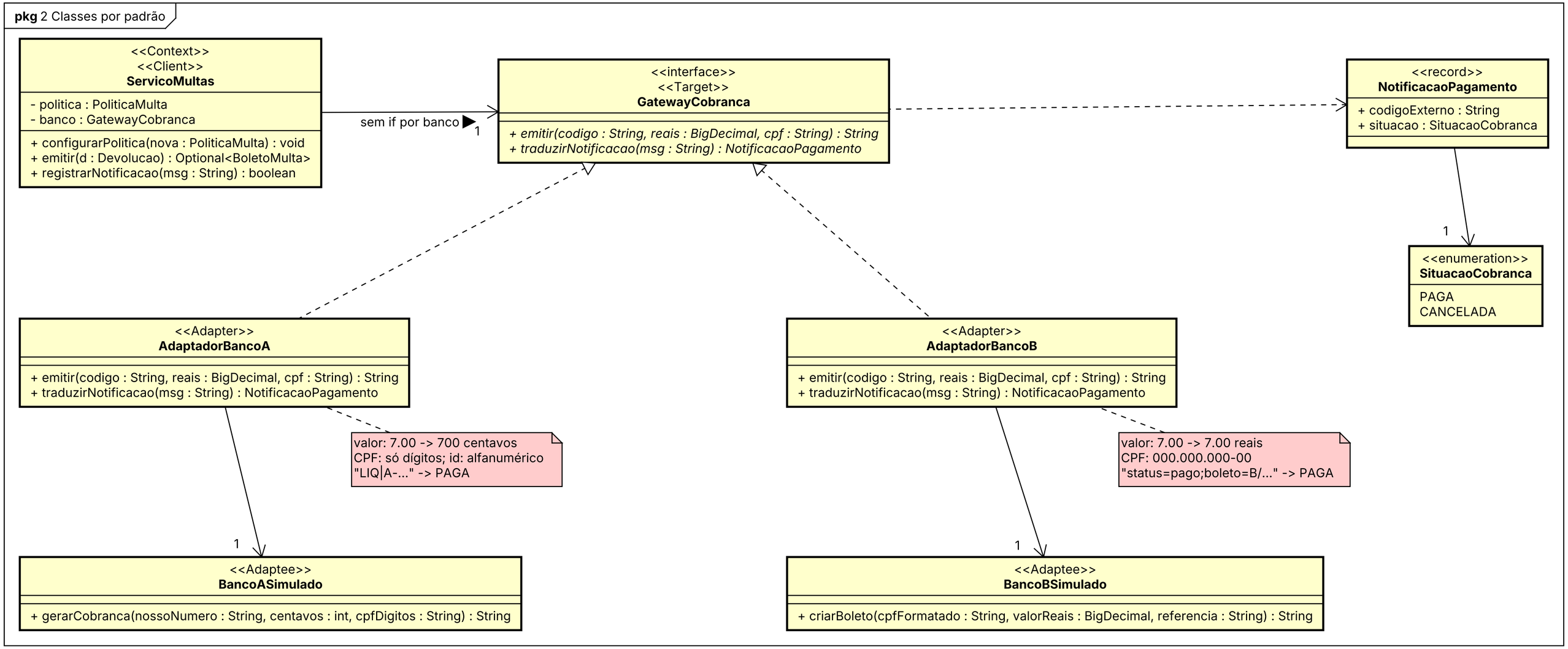
R$ 1,00 por dia

<<ConcreteStrategy>>
MultaComTeto

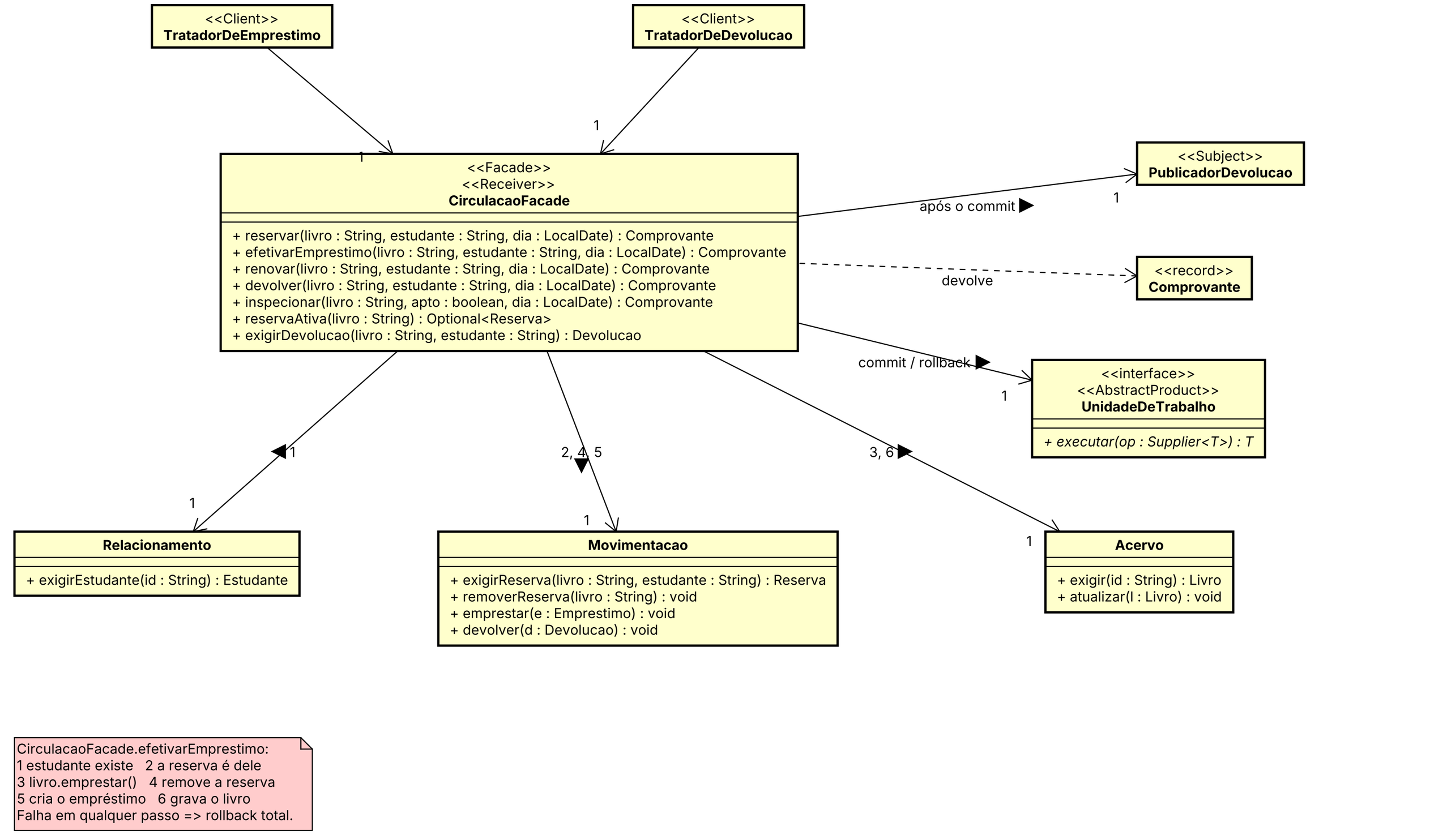
R$ 1,00/dia, máx. R$ 20,00

<<ConcreteStrategy>>
MultaComTolerancia

R$ 1,00/dia após 2 dias


pkg 2 Classes por padrão
<<Context>>
<<Client>>
ServicoMultas
- politica : PoliticaMulta
- banco : GatewayCobranca
+ configurarPolitica(nova : PoliticaMulta) : void
+ emitir(d : Devolucao) : Optional<BoletoMulta>
+ registrarNotificacao(msg : String) : boolean
sem if por banco
1
<<interface>>
<<Target>>
GatewayCobranca
+ emitir(codigo : String, reais : BigDecimal, cpf : String) : String
+ traduzirNotificacao(msg : String) : NotificacaoPagamento
<<record>>
NotificacaoPagamento
+ codigoExterno : String
+ situacao : SituacaoCobranca
1
<<enumeration>>
SituacaoCobranca
PAGA
CANCELADA
<<Adapter>>
AdaptadorBancoA
+ emitir(codigo : String, reais : BigDecimal, cpf : String) : String
+ traduzirNotificacao(msg : String) : NotificacaoPagamento
valor: 7.00 -> 700 centavos
CPF: só dígitos; id: alfanumérico
"LIQ|A-..." -> PAGA
1
<<Adapter>>
AdaptadorBancoB
+ emitir(codigo : String, reais : BigDecimal, cpf : String) : String
+ traduzirNotificacao(msg : String) : NotificacaoPagamento
valor: 7.00 -> 7.00 reais
CPF: 000.000.000-00
"status=pago;boleto=B/..." -> PAGA
1
<<Adaptee>>
BancoASimulado
+ gerarCobranca(nossoNumero : String, centavos : int, cpfDigitos : String) : String
1
<<Adaptee>>
BancoBSimulado
+ criarBoleto(cpfFormatado : String, valorReais : BigDecimal, referencia : String) : String

pkg 2 Classes por padrão



CirculacaoFacade.efetivarEmprestimo:
1 estudante existe 2 a reserva é dele
3 livro.emprestar() 4 remove a reserva
5 cria o empréstimo 6 grava o livro
Falha em qualquer passo => rollback total.

pkg 2 Classes por padrão

<<Facade>>
<<Receiver>>
CirculacaoFacade

publicar() após commit

1

<<Subject>>
PublicadorDevolucao

+ registrar(o : OuvinteDevolucao) : void
+ remover(o : OuvinteDevolucao) : void
+ publicar(ev : EventoDevolucao) : void
+ reenviarFalhas() : int
+ falhas() : List<FalhaNotificacao>

fila de reenvio

0..*

<<record>>
FalhaNotificacao

+ ouvinte : OuvinteDevolucao
+ evento : EventoDevolucao
+ erro : RuntimeException

try/catch por ouvinte: a falha de um
não impede os demais nem desfaz a devolução.

ouvintes

0..*

<<record>>
EventoDevolucao

+ devolucao : Devolucao

+ idLivro() : String
+ idEstudante() : String
+ vencimento() : LocalDate
+ dataDevolucao() : LocalDate
+ diasAtraso() : long

<<interface>>
<<Observer>>
OuvinteDevolucao

+ aoDevolver(ev : EventoDevolucao) : void

<<ConcreteObserver>>
OuvinteInspecao

+ aoDevolver(ev : EventoDevolucao) : void

<<ConcreteObserver>>
OuvinteMultas

+ aoDevolver(ev : EventoDevolucao) : void

[diasAtraso > 0] emitir

1

<<Context>>
<<Client>>
ServicoMultas

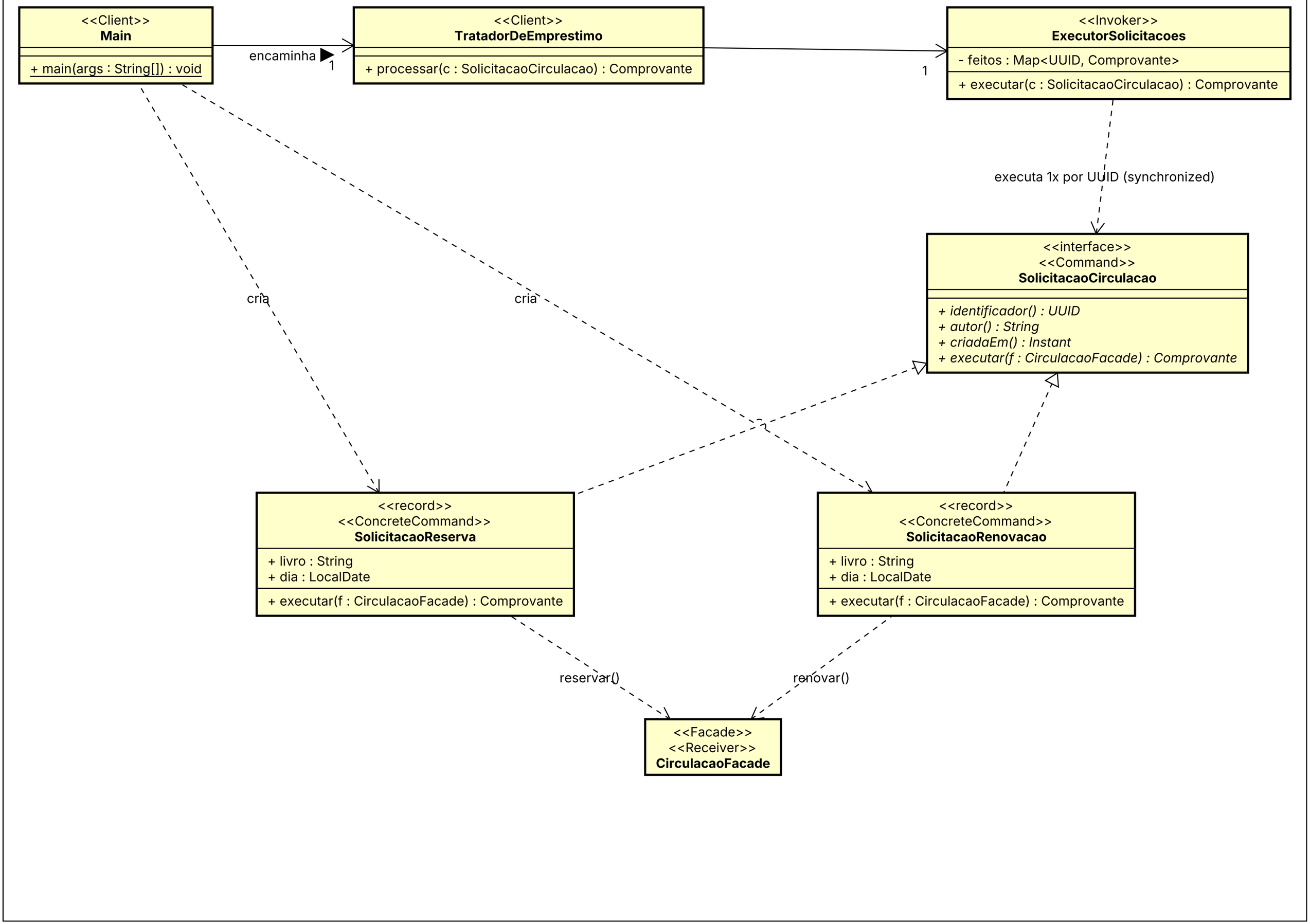
<<ConcreteObserver>>
OuvinteIndicadores

+ aoDevolver(ev : EventoDevolucao) : void

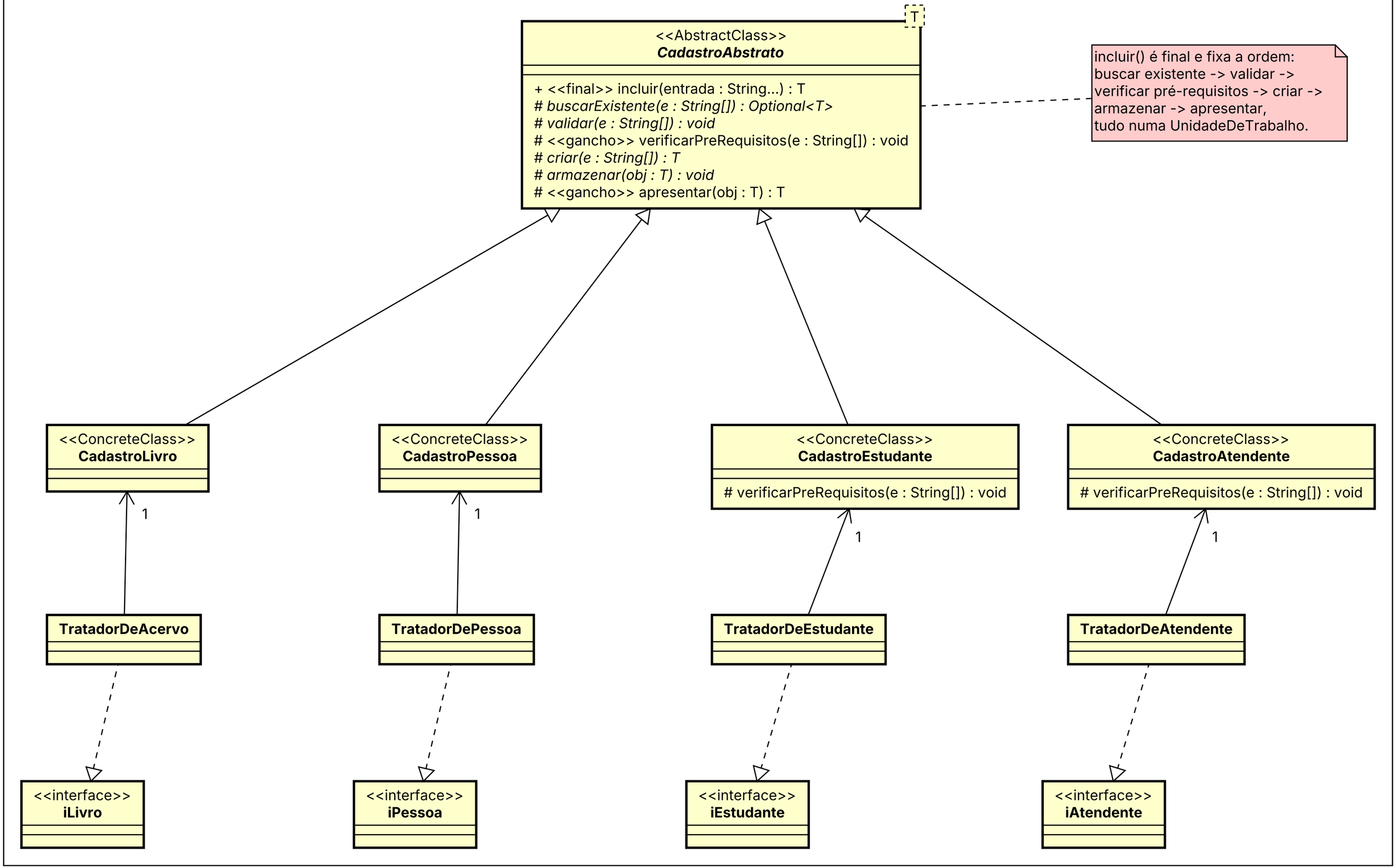
<<ConcreteObserver>>
OuvinteEmail

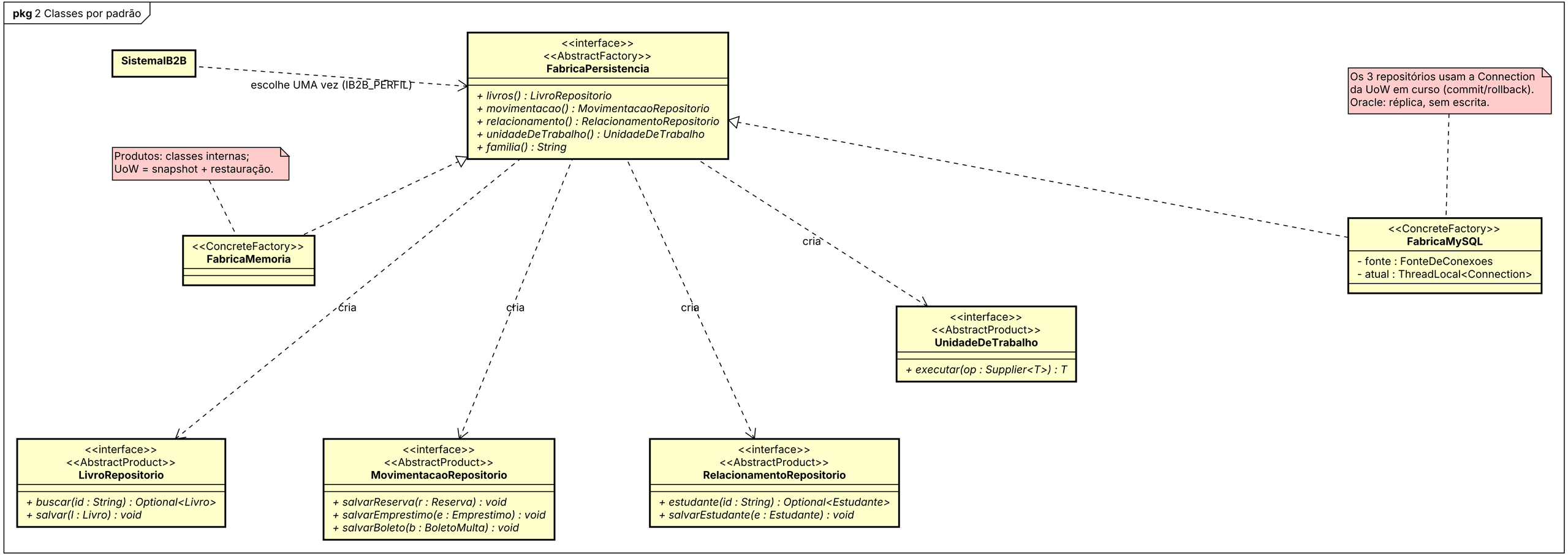
+ aoDevolver(ev : EventoDevolucao) : void

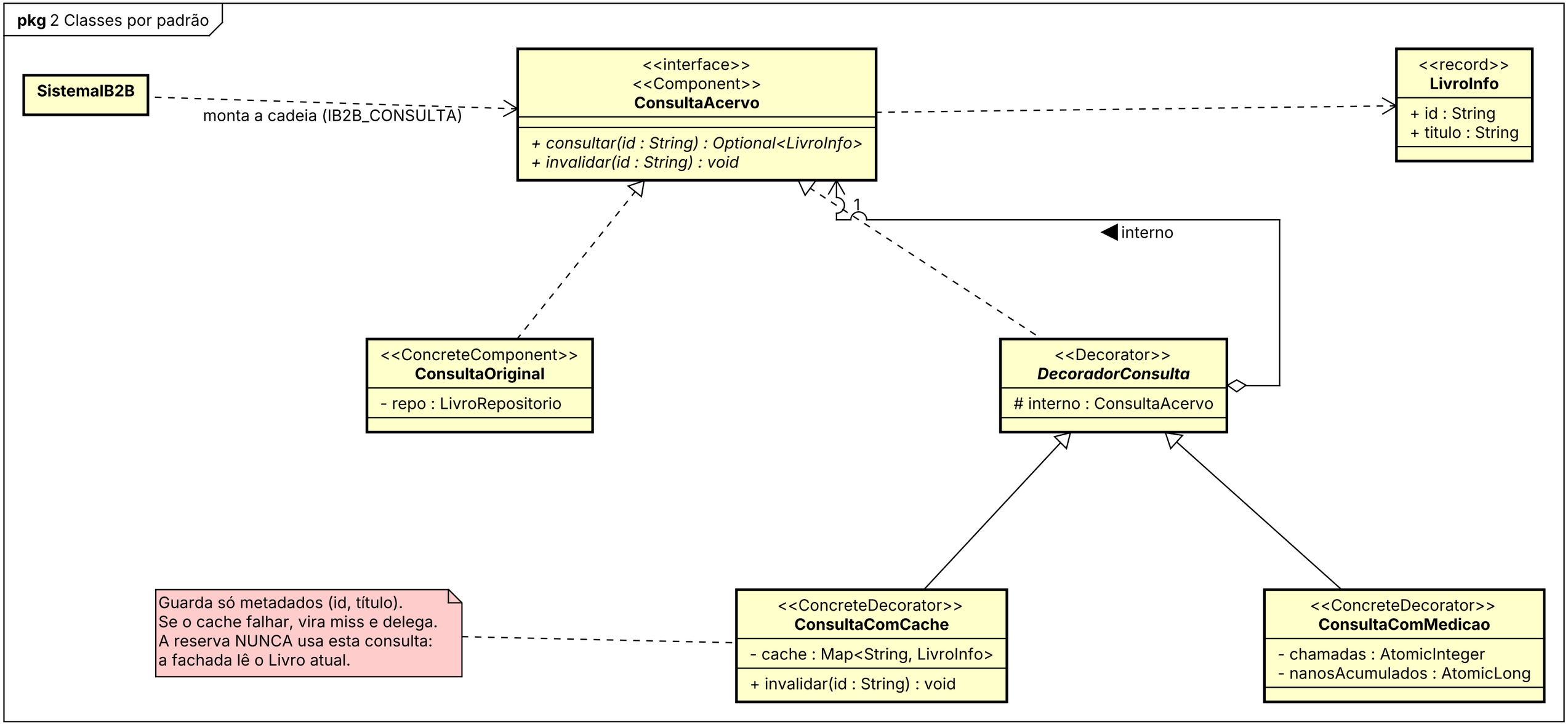
pkg 2 Classes por padrão

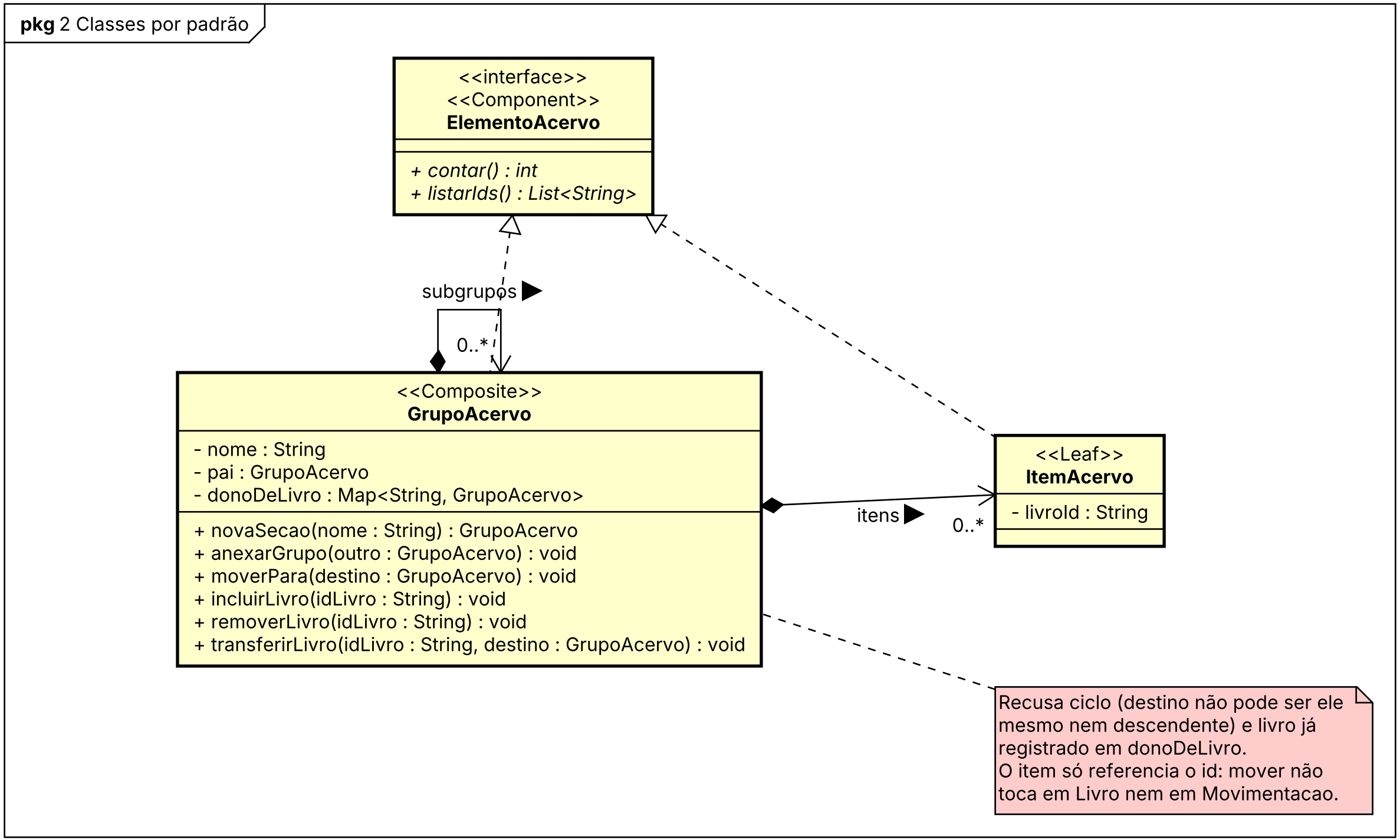


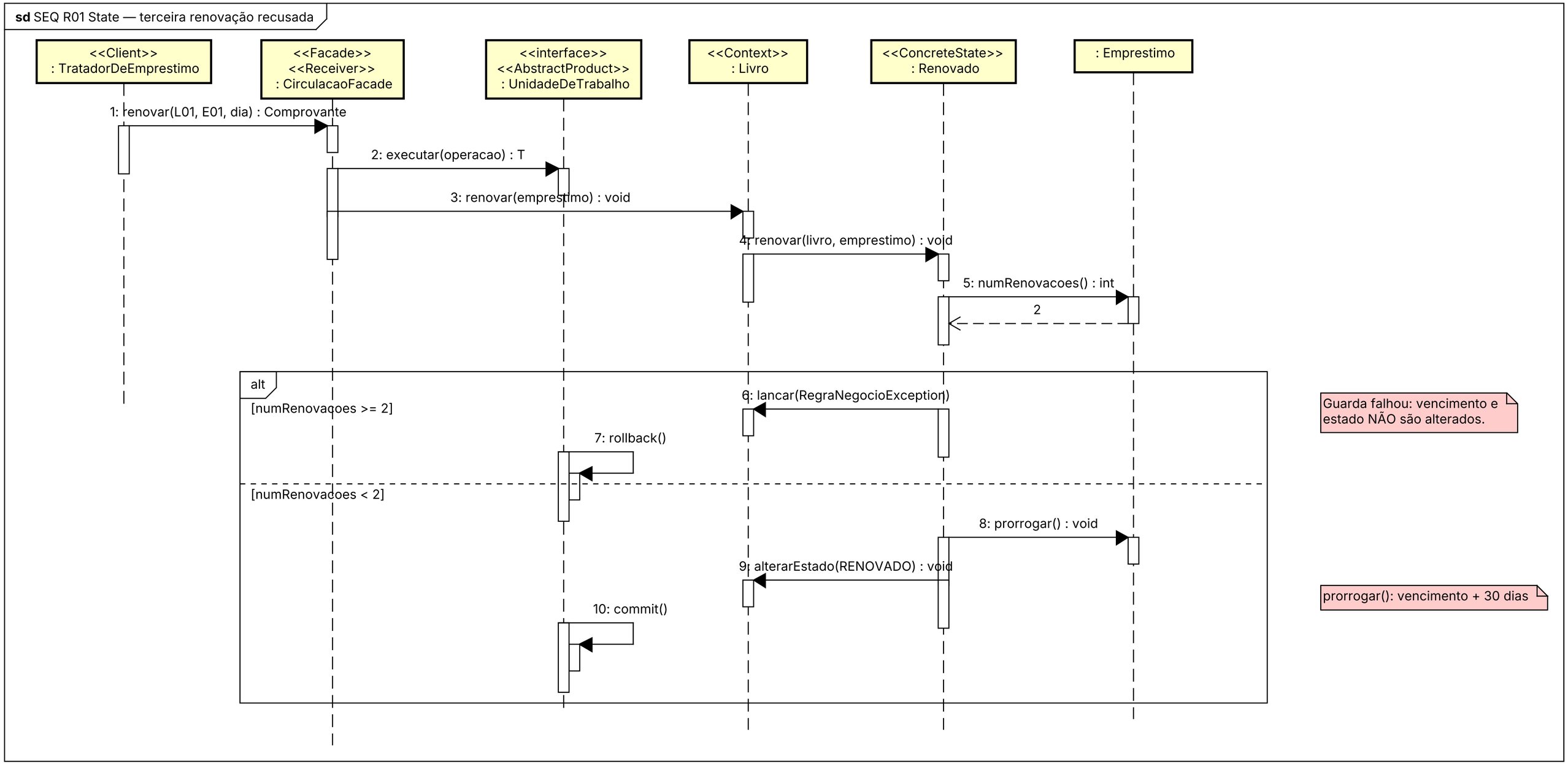
pkg 2 Classes por padrão

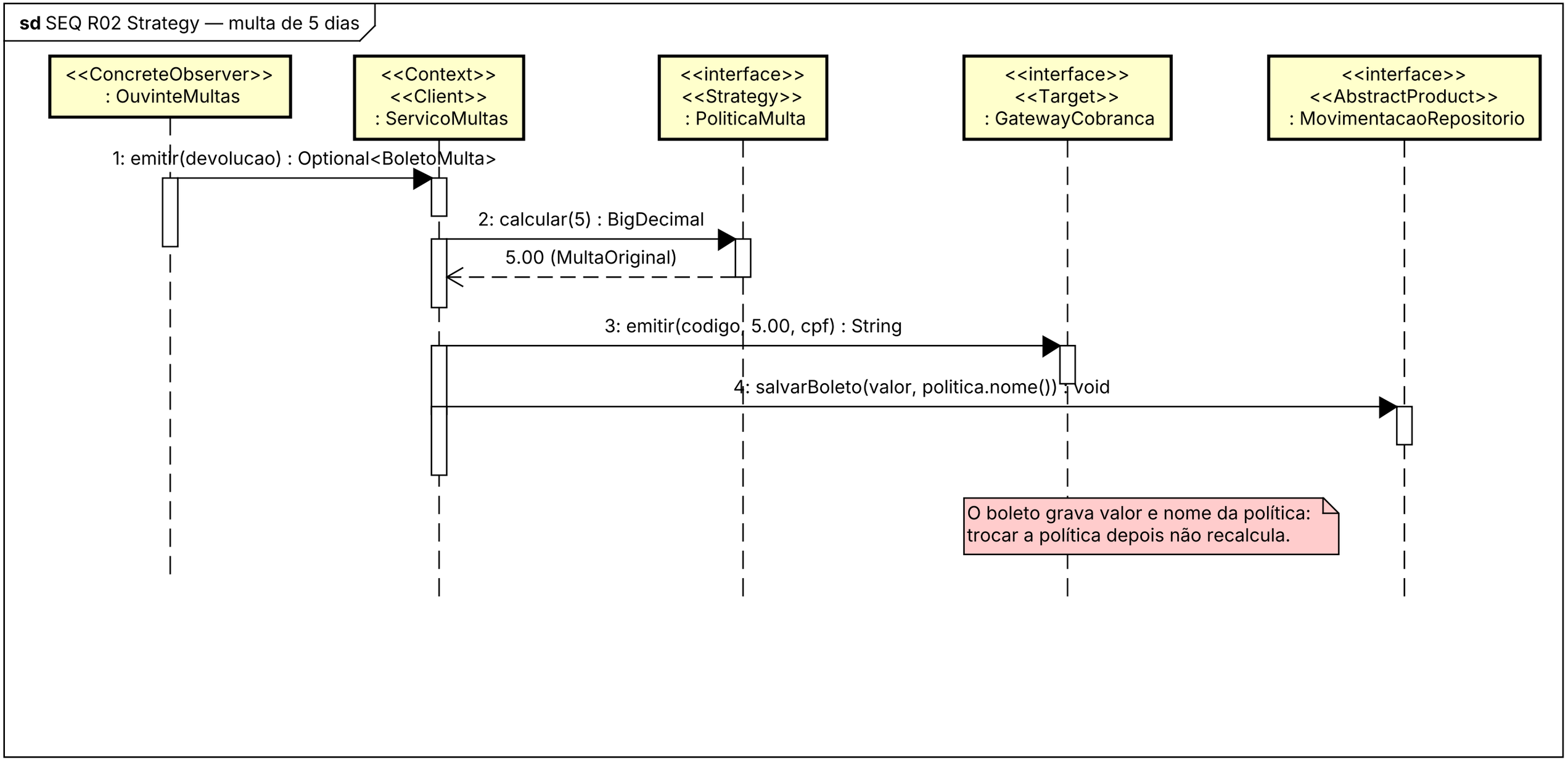



pkg 2 Classes por padrão
SistemaIB2B
escolhe UMA vez (IB2B_PERFIL)
<<interface>>
<<AbstractFactory>>
FabricaPersistencia
+ livros() : LivroRepositorio
+ movimentacao() : MovimentacaoRepositorio
+ relacionamento() : RelacionamentoRepositorio
+ unidadeDeTrabalho() : UnidadeDeTrabalho
+ familia() : String
Os 3 repositórios usam a Connection da UoW em curso (commit/rollback). Oracle: réplica, sem escrita.
Produtos: classes internas; UoW = snapshot + restauração.
<<ConcreteFactory>>
FabricaMemoria
<<ConcreteFactory>>
FabricaMySQL
- fonte : FonteDeConexoes
- atual : ThreadLocal<Connection>
cria
cria
cria
cria
<<interface>>
<<AbstractProduct>>
UnidadeDeTrabalho
+ executar(op : Supplier<T>) : T
<<interface>>
<<AbstractProduct>>
LivroRepositorio
+ buscar(id : String) : Optional<Livro>
+ salvar(l : Livro) : void
<<interface>>
<<AbstractProduct>>
MovimentacaoRepositorio
+ salvarReserva(r : Reserva) : void
+ salvarEmprestimo(e : Emprestimo) : void
+ salvarBoleto(b : BoletoMulta) : void
<<interface>>
<<AbstractProduct>>
RelacionamentoRepositorio
+ estudante(id : String) : Optional<Estudante>
+ salvarEstudante(e : Estudante) : void


pkg 2 Classes por padrão
SistemaIB2B
monta a cadeia (IB2B_CONSULTA)
<<interface>>
<<Component>>
ConsultaAcervo
+ consultar(id : String) : Optional<LivroInfo>
+ invalidar(id : String) : void
<<record>>
LivroInfo
+ id : String
+ titulo : String
1
interno
<<ConcreteComponent>>
ConsultaOriginal
- repo : LivroRepositorio
<<Decorator>>
DecoradorConsulta
interno : ConsultaAcervo
Guarda só metadados (id, título).
Se o cache falhar, vira miss e delega.
A reserva NUNCA usa esta consulta:
a fachada lê o Livro atual.
<<ConcreteDecorator>>
ConsultaComCache
- cache : Map<String, LivroInfo>
+ invalidar(id : String) : void
<<ConcreteDecorator>>
ConsultaComMedicao
- chamadas : AtomicInteger
- nanosAcumulados : AtomicLong


pkg 2 Classes por padrão
<<interface>>
<<Component>>
ElementoAcervo
+ contar() : int
+ listarIds() : List<String>
subgrupos
0..*
<<Composite>>
GrupoAcervo
- nome : String
- pai : GrupoAcervo
- donoDeLivro : Map<String, GrupoAcervo>
+ novaSecao(nome : String) : GrupoAcervo
+ anexarGrupo(outro : GrupoAcervo) : void
+ moverPara(destino : GrupoAcervo) : void
+ incluirLivro(idLivro : String) : void
+ removerLivro(idLivro : String) : void
+ transferirLivro(idLivro : String, destino : GrupoAcervo) : void
itens
0..*
<<Leaf>>
ItemAcervo
- livroId : String
Recusa ciclo (destino não pode ser ele mesmo nem descendente) e livro já registrado em donoDeLivro.
O item só referencia o id: mover não toca em Livro nem em Movimentacao.


sd SEQ R01 State — terceira renovação recusada
<<Client>>
: TratadorDeEmprestimo
<<Facade>>
<<Receiver>>
: CirculacaoFacade
<<interface>>
<<AbstractProduct>>
: UnidadeDeTrabalho
<<Context>>
: Livro
<<ConcreteState>>
: Renovado
: Emprestimo
1: renovar(L01, E01, dia) : Comprovante
2: executar(operacao) : T
3: renovar(emprestimo) : void
4: renovar(livro, emprestimo) : void
5: numRenovacoes() : int
2
alt
[numRenovacoes >= 2]
6: lancar(RegraNegocioException)
7: rollback()
[numRenovacoes < 2]
8: prorrogar() : void
9: alterarEstado(RENOVADO) : void
10: commit()
Guarda falhou: vencimento e estado NÃO são alterados.
prorrogar(): vencimento + 30 dias


sd SEQ R02 Strategy — multa de 5 dias
<<ConcreteObserver>>
: OuvinteMultas
<<Context>>
<<Client>>
: ServicoMultas
<<interface>>
<<Strategy>>
: PoliticaMulta
<<interface>>
<<Target>>
: GatewayCobranca
<<interface>>
<<AbstractProduct>>
: MovimentacaoRepositorio
1: emitir(devolucao) : Optional<BoletoMulta>
2: calcular(5) : BigDecimal
5.00 (MultaOriginal)
3: emitir(codigo, 5.00, cpf) : String
4: salvarBoleto(valor, politica.nome()) : void
O boleto grava valor e nome da política:
trocar a política depois não recalcula.

sd SEQ R03 Adapter — R$ 7,00 nos bancos A e B

<<Context>>
<<Client>>
: ServicoMultas

<<interface>>
<<Target>>
: GatewayCobranca

<<Adapter>>
: AdaptadorBancoA

<<Adaptee>>
: BancoASimulado

<<Adapter>>
: AdaptadorBancoB

<<Adaptee>>
: BancoBSimulado

1: [instalação com banco A] emitir("M-L01", 7.00, cpf) : String

2: emitir("M-L01", 7.00, cpf) : String

3: gerarCobranca("ML01", 700, "12345678901") : String

"A-ML01"

4: traduzirNotificacao("LIQ|A-ML01") : NotificacaoPagamento

5: traduzirNotificacao("LIQ|A-ML01") : NotificacaoPagamento

NotificacaoPagamento(A-ML01, PAGA)

6: quitar()

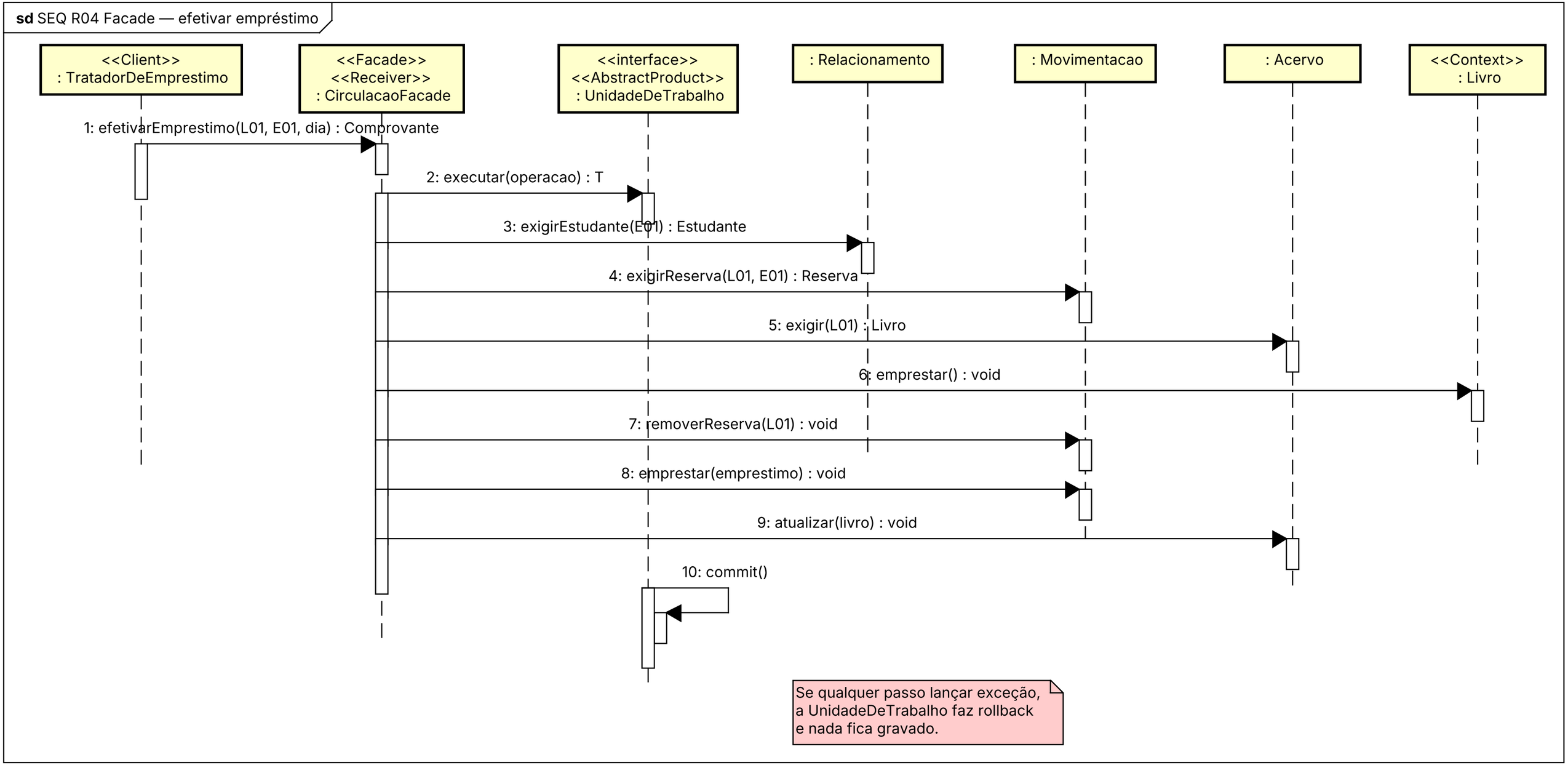
7: [instalação com banco B] emitir("M-L01", 7.00, cpf) : String

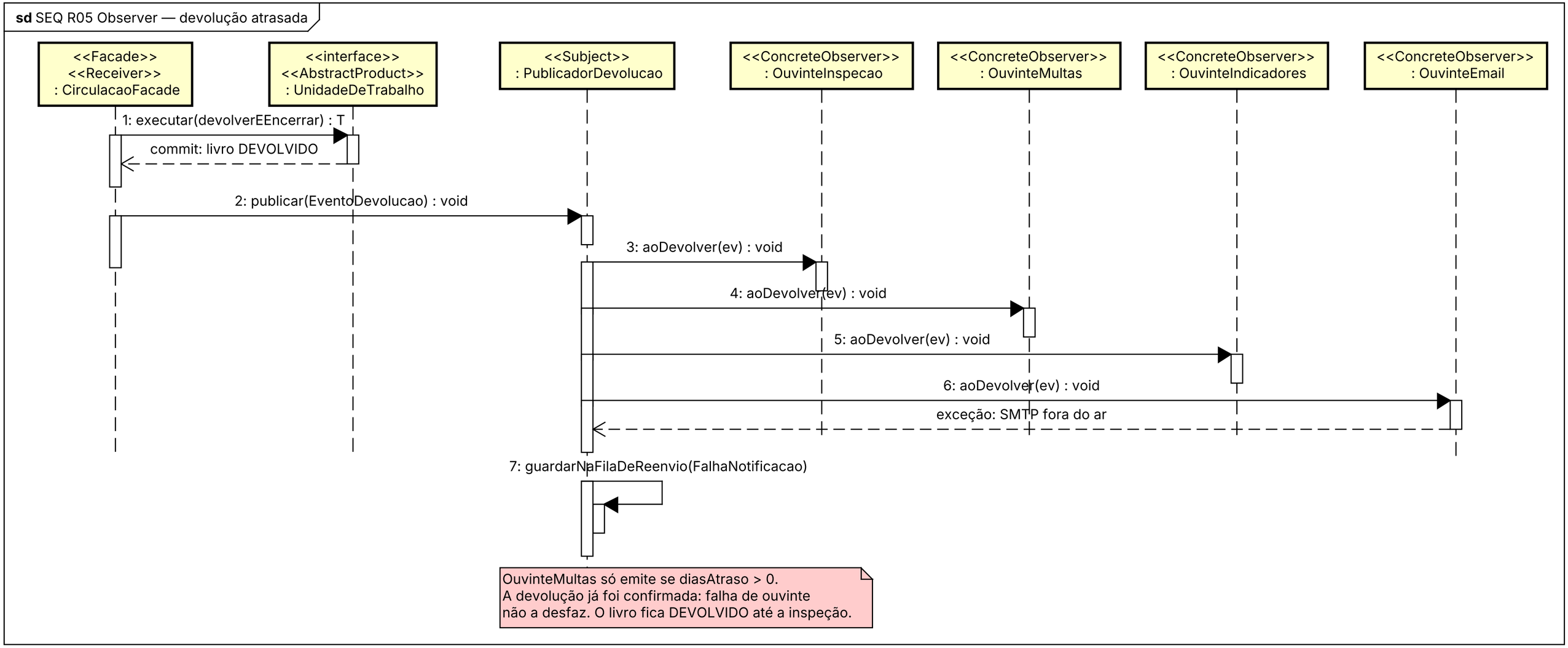
8: emitir("M-L01", 7.00, cpf) : String

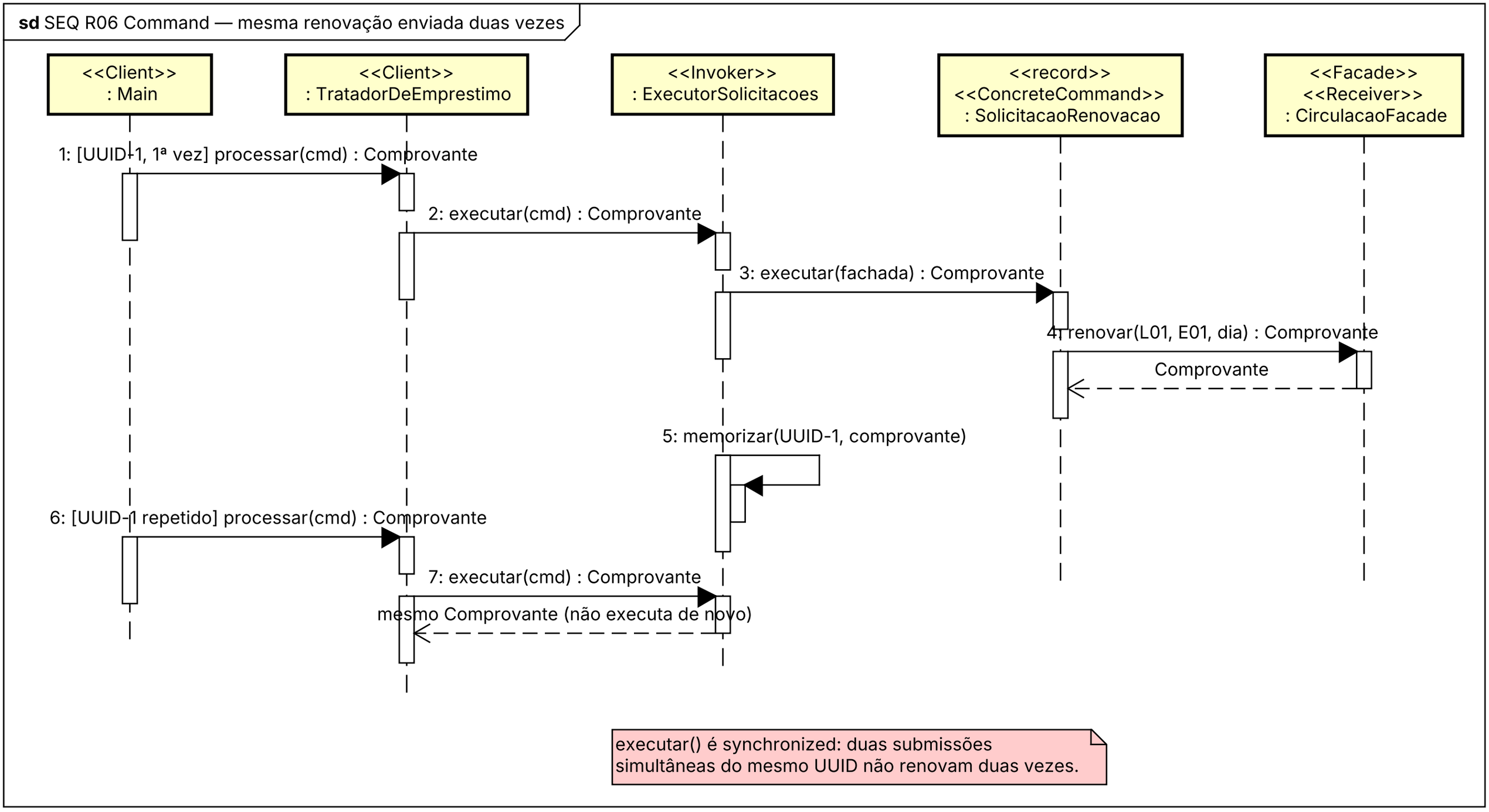
9: criarBoleto("123.456.789-01", 7.00, "M-L01") : String

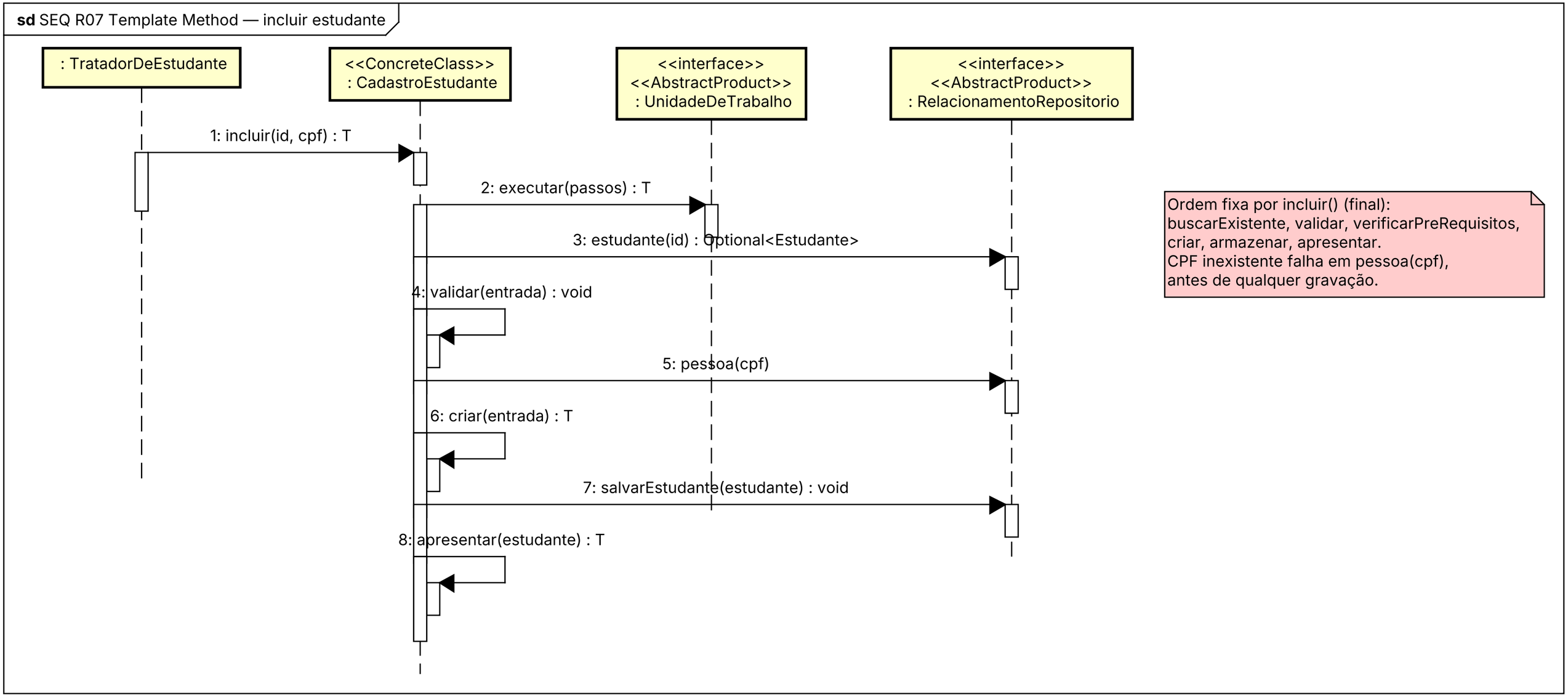
"B/M-L01"

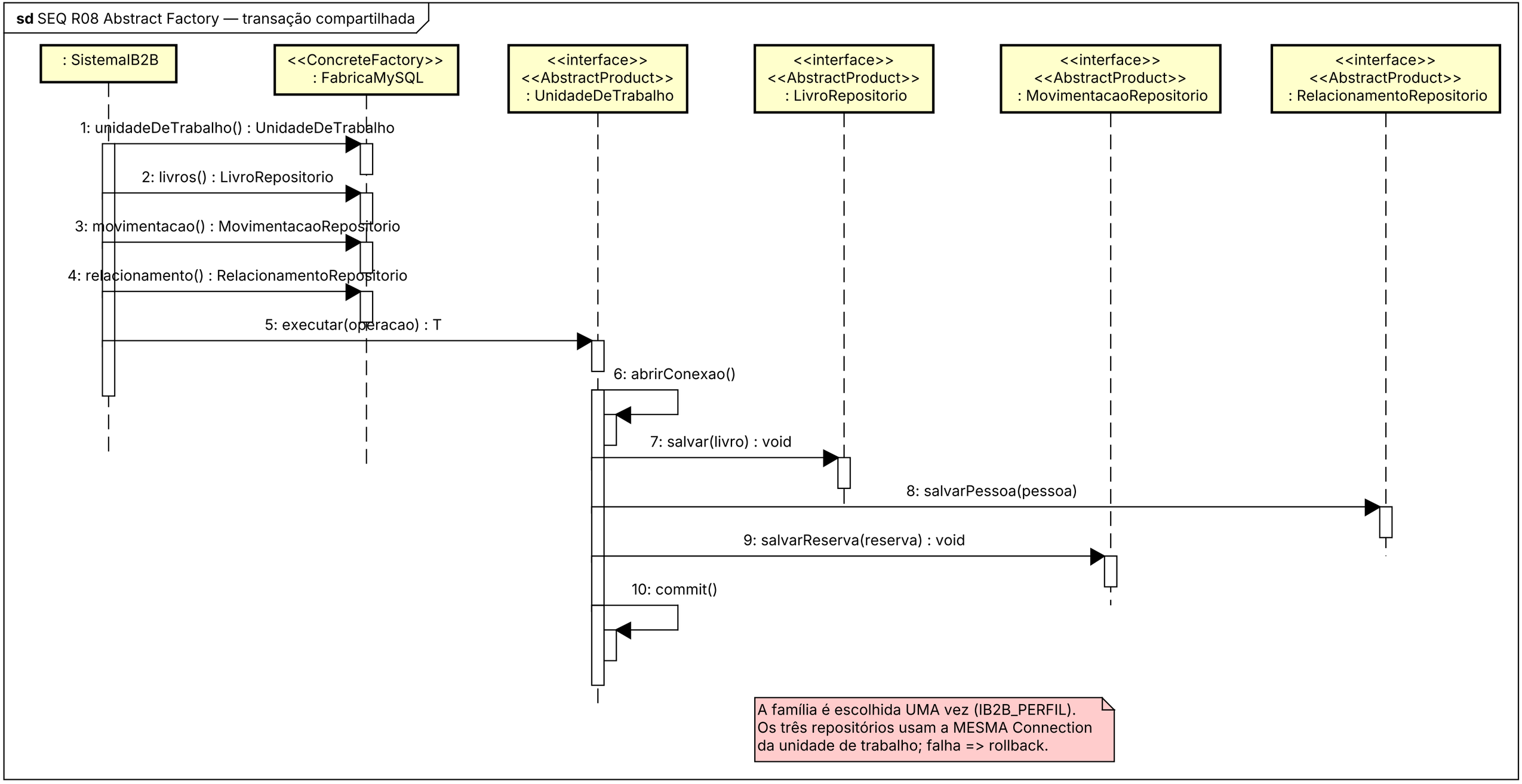
quitar() devolve false numa notificação repetida: não há segunda quitação.

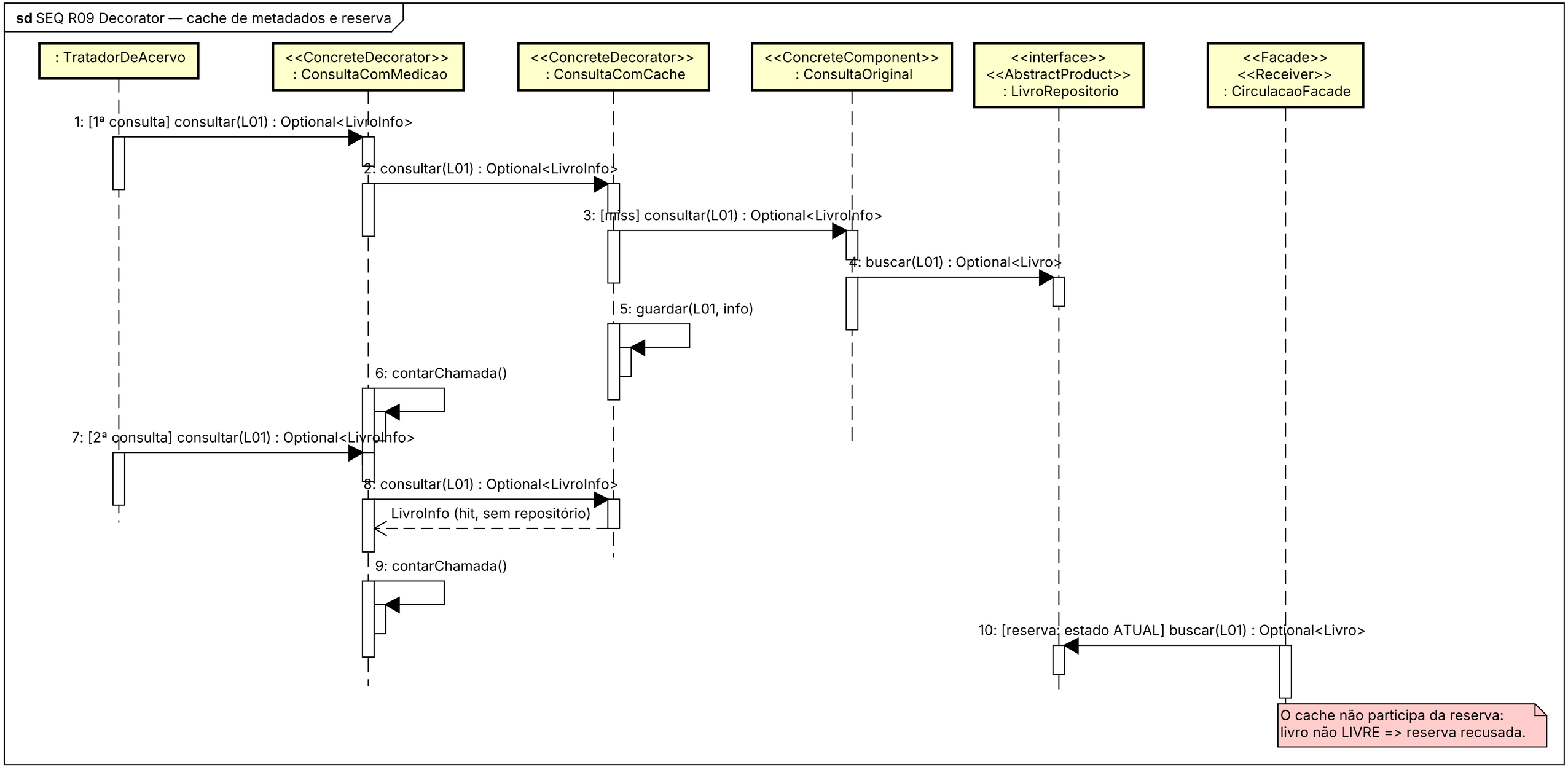

sd SEQ R04 Facade — efetivar empréstimo
<<Client>>
: TratadorDeEmprestimo
<<Facade>>
<<Receiver>>
: CirculacaoFacade
<<interface>>
<<AbstractProduct>>
: UnidadeDeTrabalho
: Relacionamento
: Movimentacao
: Acervo
<<Context>>
: Livro
1: efetivarEmprestimo(L01, E01, dia) : Comprovante
2: executar(operacao) : T
3: exigirEstudante(E01) : Estudante
4: exigirReserva(L01, E01) : Reserva
5: exigir(L01) : Livro
6: emprestar() : void
7: removerReserva(L01) : void
8: emprestar(emprestimo) : void
9: atualizar(livro) : void
10: commit()
Se qualquer passo lançar exceção,
a UnidadeDeTrabalho faz rollback
e nada fica gravado.

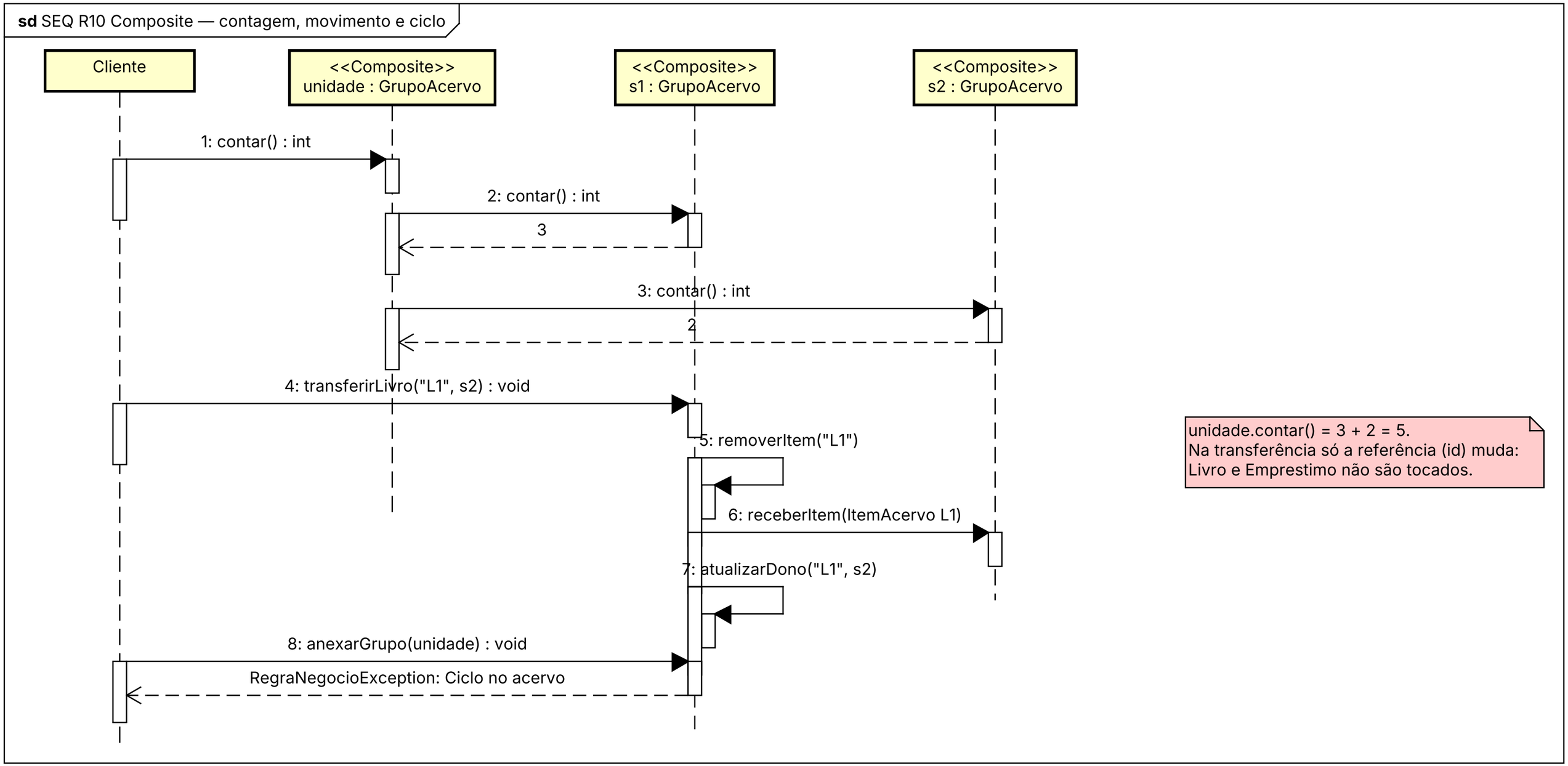

sd SEQ R05 Observer — devolução atrasada
<<Facade>>
<<Receiver>>
: CirculacaoFacade
<<interface>>
<<AbstractProduct>>
: UnidadeDeTrabalho
<<Subject>>
: PublicadorDevolucao
<<ConcreteObserver>>
: OuvinteInspecao
<<ConcreteObserver>>
: OuvinteMultas
<<ConcreteObserver>>
: OuvinteIndicadores
<<ConcreteObserver>>
: OuvinteEmail
1: executar(devolverEEncerrar) : T
commit: livro DEVOLVIDO
2: publicar(EventoDevolucao) : void
3: aoDevolver(ev) : void
4: aoDevolver(ev) : void
5: aoDevolver(ev) : void
6: aoDevolver(ev) : void
exceção: SMTP fora do ar
7: guardarNaFilaDeReenvio(FalhaNotificacao)
OuvinteMultas só emite se diasAtraso > 0.
A devolução já foi confirmada: falha de ouvinte
não a desfaz. O livro fica DEVOLVIDO até a inspeção.

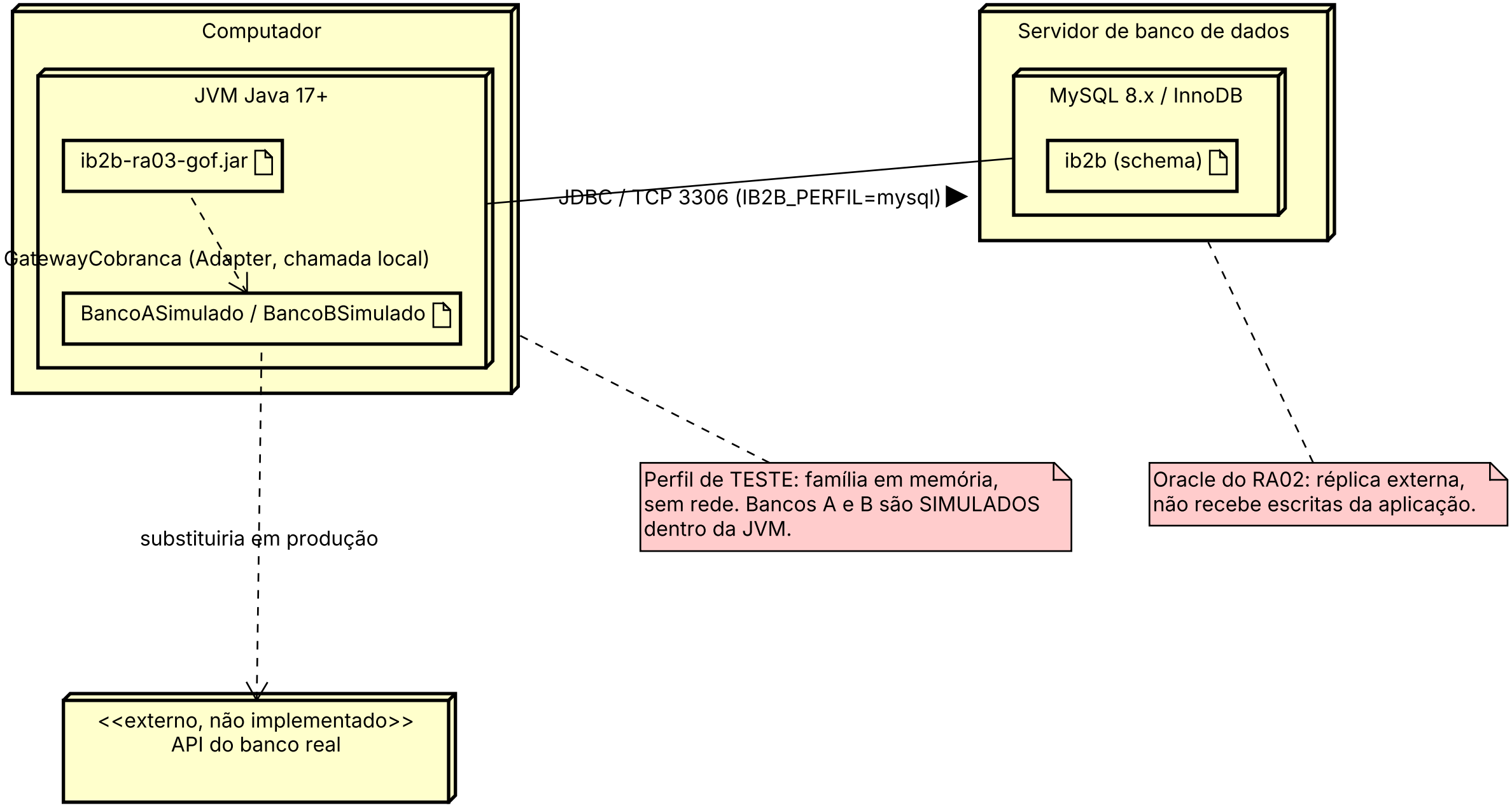

sd SEQ R06 Command — mesma renovação enviada duas vezes
<<Client>>
: Main
<<Client>>
: TratadorDeEmprestimo
<<Invoker>>
: ExecutorSolicitacoes
<<record>>
<<ConcreteCommand>>
: SolicitacaoRenovacao
<<Facade>>
<<Receiver>>
: CirculacaoFacade
1: [UUID-1, 1ª vez] processar(cmd) : Comprovante
2: executar(cmd) : Comprovante
3: executar(fachada) : Comprovante
4: renovar(L01, E01, dia) : Comprovante
Comprovante
5: memorizar(UUID-1, comprovante)
6: [UUID-1 repetido] processar(cmd) : Comprovante
7: executar(cmd) : Comprovante
mesmo Comprovante (não executa de novo)
executar() é synchronized: duas submissões
simultâneas do mesmo UUID não renovam duas vezes.

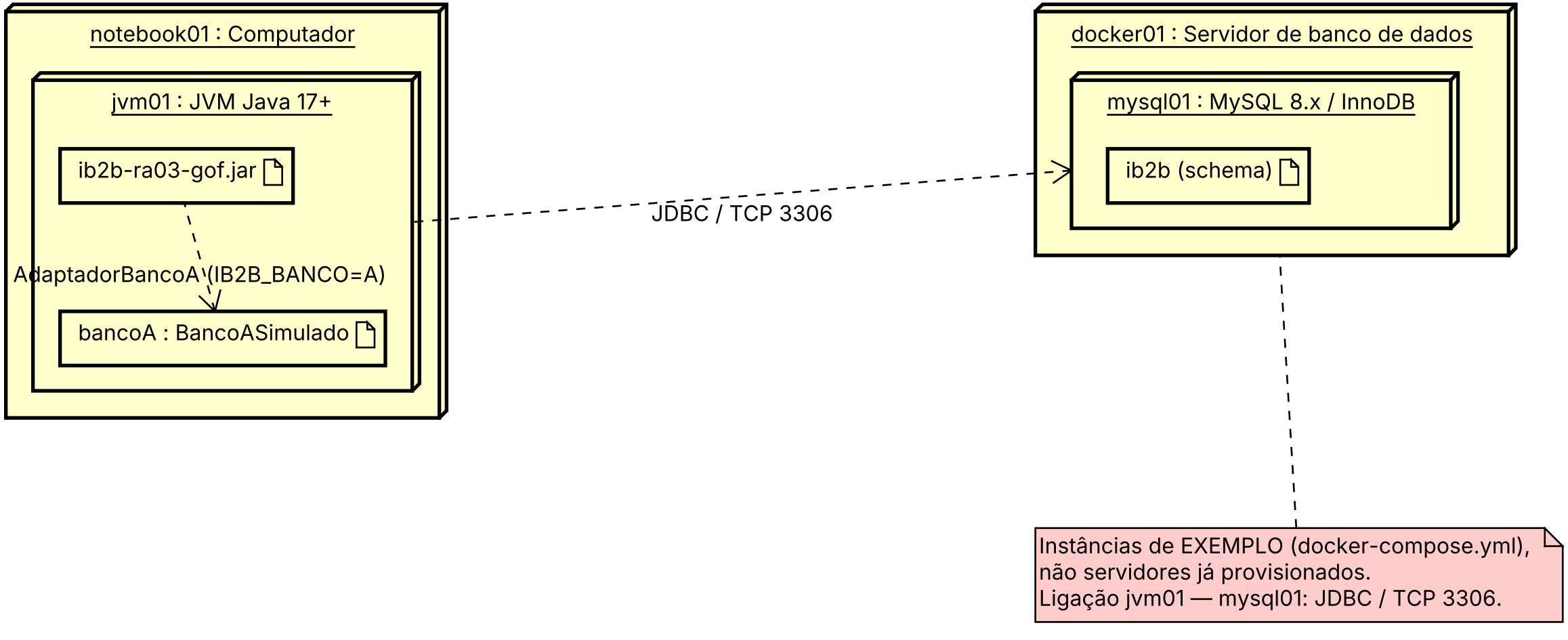

sd SEQ R07 Template Method — incluir estudante
: TratadorDeEstudante
<<ConcreteClass>>
: CadastroEstudante
<<interface>>
<<AbstractProduct>>
: UnidadeDeTrabalho
<<interface>>
<<AbstractProduct>>
: RelacionamentoRepositorio
1: incluir(id, cpf) : T
2: executar(passos) : T
3: estudante(id) : Optional<Estudante>
4: validar(entrada) : void
5: pessoa(cpf)
6: criar(entrada) : T
7: salvarEstudante(estudante) : void
8: apresentar(estudante) : T
Ordem fixa por incluir() (final):
buscarExistente, validar, verificarPreRequisitos,
criar, armazenar, apresentar.
CPF inexistente falha em pessoa(cpf),
antes de qualquer gravação.


sd SEQ R08 Abstract Factory — transação compartilhada
: SistemaIB2B
<<ConcreteFactory>>
: FabricaMySQL
<<interface>>
<<AbstractProduct>>
: UnidadeDeTrabalho
<<interface>>
<<AbstractProduct>>
: LivroRepositorio
<<interface>>
<<AbstractProduct>>
: MovimentacaoRepositorio
<<interface>>
<<AbstractProduct>>
: RelacionamentoRepositorio
1: unidadeDeTrabalho() : UnidadeDeTrabalho
2: livros() : LivroRepositorio
3: movimentacao() : MovimentacaoRepositorio
4: relacionamento() : RelacionamentoRepositorio
5: executar(operacao) : T
6: abrirConexao()
7: salvar(livro) : void
8: salvarPessoa(pessoa)
9: salvarReserva(reserva) : void
10: commit()
A família é escolhida UMA vez (IB2B_PERFIL).
Os três repositórios usam a MESMA Connection
da unidade de trabalho; falha => rollback.


sd SEQ R09 Decorator — cache de metadados e reserva
: TratadorDeAcervo
<<ConcreteDecorator>>
: ConsultaComMedicao
<<ConcreteDecorator>>
: ConsultaComCache
<<ConcreteComponent>>
: ConsultaOriginal
<<interface>>
<<AbstractProduct>>
: LivroRepositorio
<<Facade>>
<<Receiver>>
: CirculacaoFacade
1: [1ª consulta] consultar(L01) : Optional<LivroInfo>
2: consultar(L01) : Optional<LivroInfo>
3: [miss] consultar(L01) : Optional<LivroInfo>
4: buscar(L01) : Optional<Livro>
5: guardar(L01, info)
6: contarChamada()
7: [2ª consulta] consultar(L01) : Optional<LivroInfo>
8: consultar(L01) : Optional<LivroInfo>
LivroInfo (hit, sem repositório)
9: contarChamada()
10: [reserva: estado ATUAL] buscar(L01) : Optional<Livro>
O cache não participa da reserva:
livro não LIVRE => reserva recusada.


sd SEQ R10 Composite — contagem, movimento e ciclo
Cliente
<<Composite>>
unidade : GrupoAcervo
<<Composite>>
s1 : GrupoAcervo
<<Composite>>
s2 : GrupoAcervo
1: contar() : int
2: contar() : int
3
3: contar() : int
2
4: transferirLivro("L1", s2) : void
5: removerItem("L1")
6: receberItem(ItemAcervo L1)
7: atualizarDono("L1", s2)
8: anexarGrupo(unidade) : void
RegraNegocioException: Ciclo no acervo
unidade.contar() = 3 + 2 = 5.
Na transferência só a referência (id) muda:
Livro e Emprestimo não são tocados.


Computador
JVM Java 17+
ib2b-ra03-gof.jar
GatewayCobranca (Adapter, chamada local)
BancoASimulado / BancoBSimulado
Servidor de banco de dados
MySQL 8.x / InnoDB
ib2b (schema)
JDBC / TCP 3306 (IB2B_PERFIL=mysql)
Perfil de TESTE: família em memória,
sem rede. Bancos A e B são SIMULADOS
dentro da JVM.
Oracle do RA02: réplica externa,
não recebe escritas da aplicação.
substituiria em produção
<<externo, não implementado>>
API do banco real


notebook01 : Computador
jvm01 : JVM Java 17+
ib2b-ra03-gof.jar
AdaptadorBancoA (IB2B_BANCO=A)
bancoA : BancoASimulado
JDBC / TCP 3306
docker01 : Servidor de banco de dados
mysql01 : MySQL 8.x / InnoDB
ib2b (schema)
Instâncias de EXEMPLO (docker-compose.yml),
não servidores já provisionados.
Ligação jvm01 — mysql01: JDBC / TCP 3306.